

Project assignment

ORM

Contents

- Data persistence and Direct data access
- What is ORM
- ORM - Entities
- ORM - Sessions & Transactions
- The Repository Pattern
- Designing a Layered ORM-Based Application

Data persistence and Direct data access

Persistence

Salvador Dalí (Figueres, 1904 – Figueres 1989)
La persistencia de la memoria
1931
Oil on canvas
24 × 33 cm
Museum of Modern Art (MoMA), New York



Persistence

- Persistence simply means that we would like our program's data to outlive the current process.
- For Java developers, this means we would like the state of certain objects to live beyond the scope of the JVM,
 - so that the same state is available in a different JVM,
 - or even in a different program written in a different programming language.

Persistent vs. In-Memory Data

- Need for persistence: Data must survive program execution
- In-memory data: Stored in RAM, lost when the program stops
- Persistent data: Stored on disk, databases, or external storage
- Key differences: Volatility, lifespan, and accessibility

Direct Data Access - Definition

Direct data access can be defined as the practice of interacting with a database without using any abstraction layer, relying only on a language native database driver to connect to the database, execute queries and manage transactions.

Direct Data Access – Key features

- SQL queries always embedded directly in the source code
- Strong dependency on the database structure and SQL dialect
- Strong coupling between application code and database schema
- Requires explicit management of connections, transactions, and error handling

Limitations & Challenges

- Risk of code duplication
- Prone to maintenance issues as database schema evolves
- Risk of SQL injection if queries are not properly sanitized
- Portability issues & vendor lock-in: SQL dialects may differ between databases, so changing DBMS can be difficult
- Managing connections manually can lead to performance and scalability issues

What is ORM

Object/Relational Mismatch

- Impedance mismatch refers to differences in relational data model compared object-oriented paradigm
 - The relational model represents data in a tabular format
 - Object-oriented languages represent data as an interconnected graph of objects with inheritance and polymorphism.

Object/Relational Mismatch

- Granularity
- Subtypes (inheritance)
- Identity
- Associations
- Data navigation

Object/Relational Mismatch

- Granularity
 - Object models often have more classes than tables in the database.
- Subtypes (inheritance)
 - The relational model doesn't have inheritance as a first-class construct,
 - There are standard ways to represent subtyping.
- Identity
 - The relational model defines exactly one notion of "sameness": the primary key.
 - Java defines both object identity **`a==b`** and value equality **`a.equals(b)`**.
 - It is common to have more than one object in a given Java program representing the same row of a database table.

Object/Relational Mismatch

- Associations
 - Associations are represented as unidirectional references in Java
 - Relational databases treat all associations as bidirectional and represent them via foreign keys
 - Bidirectional relationship in Java require code on both sides of the association
 - Just by looking at the Java code we cannot be sure of the multiplicity of an association
- Data navigation
 - In Java, we navigate associations from one object to the next, walking the object network.
 - That is inefficient when retrieving relational data
 - The goals is to minimize the number of SQL queries

ORM - Definition

- Object-Relational Mapping (ORM) is a technique for managing database interactions using object-oriented programming, converting data between a relational database and the memory

ORM – Key Features

- Provides an abstraction layer that decouples business logic from database access
- Translates objects in the application into database records and vice versa
- Automates data conversion between object models and relational schemas
- Reduces (in most cases eliminates) the need for direct SQL queries for basic operations

ORM – Key Features

- Automates query generation and result mapping to objects
- Reduces manual connection and transaction management

ORM vs Direct Data Access

- Direct Data Access
 - Uses native SQL to interact with the database
 - Directly embeds SQL in the application code
 - Manually handles every relationship and transaction
- ORM
 - Uses object-based operations to interact with the database
 - SQL queries can be automatically generated based on object models
 - Simplifies complex relationships and data handling

ORM – Database connection

- An application must establish a connection before interacting with a database
- ORM frameworks abstract connection handling, reducing manual configuration
- Connection settings typically include:
 - Database location (host, port, or file)
 - Authentication credentials
 - SQL dialect
 - Performance optimizations settings (e.g., caching, connection pooling)

ORM – SQL Dialect

- ORMs provide an abstraction layer over database-specific SQL syntax
- Still, different databases have variations in SQL syntax and behavior
- Specifying an SQL dialect allows the ORM to generate queries compatible with the chosen database
- The ORM adapts automatically, reducing vendor lock-in and improving portability
- Applications can switch databases with a minimal configuration change

ORM – Pros & Cons

- PRO
 - Reduces the need to insert SQL within our programming language
 - Improves code maintainability and readability
 - Reduces development time by automating database interactions
 - Supports multiple databases with minimal changes

ORM – Pros & Cons

- CON
 - Less control over query optimization and performance (but it is possible to use direct SQL queries)
 - Potential overhead compared to manually optimized SQL
 - Learning curve for ORM frameworks and configurations

ORM - Entities

Entity – Definition

An entity is a purely structural component that represents a persistent object that is mapped to a database table. It is a fundamental building block of the application's domain model and serves as an abstraction over relational database records.

Entity – Key Features

- Entities encapsulate only data structure and constraints, without containing business logic
- Each entity instance corresponds to a row in the table
- Defines fields (mapped to table columns) and relationships with other entities
- Must have at least one primary key to ensure uniqueness
- By defining entities, we model the ***granularity*** of the object domain, ensuring a structured representation of data in the application

Entity – Fields Mapping

- Each entity field corresponds to a database column
- Data type conversion between object model and relational model
 - String in code → VARCHAR in database
 - boolean in code → TINYINT(1) in some databases
- Handling type mismatches
 - Some languages support richer types than relational databases
 - ORM provides mapping rules for correct conversion
- Custom type mapping possible if default conversions are not sufficient

Entity – Fields Attributes

- For each entity field it is possible to define the constraints and properties typical of a relational database schema
- Primary keys attribute
 - Establish the ***identity*** of an entity instance
 - Define entity uniqueness
 - Auto-generated (e.g., auto-increment, UUID) or manually assigned
 - A primary key is always unique and non-null

Entity – Fields Attributes

- Field generic attributes
 - Foreign key: The field references a primary key from another table
 - Nullable vs. Non-nullable: Whether a field must always have a value
 - Unique constraints: Prevent duplicate values in a column
 - Default values: Automatically assigned if no value is provided
 - Data length constraints: Enforce maximum storage size (e.g., VARCHAR(255))

Entity – Inheritance Mapping

- Relational databases do not support **subtypes (inheritance)** natively, so ORM provides different strategies to map class hierarchies
- Three common mapping strategies:
 - Single Table Inheritance – One table stores all subclasses, with a type discriminator column
 - Joined Table Inheritance – Each class has its own table, linked through foreign keys
 - Table Per Class – Each class has a separate table with all its attributes
- Trade-offs:
 - Single Table is more efficient but wastes space
 - Joined Table normalizes data but adds complexity
 - Table Per Class avoids duplication but makes querying harder

Entity – Relationship Mapping

- Relationships define the ***associations*** of our model, how entities are linked in a relational model
- Four main types of relationships:
 - One-to-One (1:1) – Each entity instance maps to exactly one related entity
 - One-to-Many (1:N) – One entity is related to multiple entities
 - Many-to-One (N:1) – Multiple entities reference the same related entity
 - Many-to-Many (N:N) – Requires a join table to establish the relationship
- Foreign keys are used to represent relationships at the database level
- Associations in ORM can be unidirectional or bidirectional

Entity – Relationship Cascading

- Cascading allows propagation of operations (e.g., persist, update, delete) from a parent entity to its related entities.
- ORMs let you configure cascading per operation type:
 - Persist: saves related entities when the parent is saved.
 - Merge: updates related entities when the parent is updated.
 - Remove: deletes related entities when the parent is deleted.
 - Detach: detaches related entities when the parent is removed from the persistence context.
 - Refresh: reloads related entities when the parent is refreshed.
- Cascade strategies:
 - Full cascade: applies all operations to related entities.
 - Selective cascade: allows only specific operations to be propagated.
 - No cascade: relationships must be managed manually.
- Cascading simplifies persistence management but must be configured carefully to avoid unintended side effects.

Entity – Loading strategies

- The loading strategy determines the ***data navigation*** in our model
- Navigating relationships in ORM requires fetching data efficiently
- Two primary data loading strategies:
 - Lazy Loading – Related data is only loaded when explicitly accessed
 - Eager Loading – Related data is loaded immediately with the entity
- Pros and cons:
 - Lazy Loading improves initial performance but may cause multiple queries
 - Eager Loading reduces query count but can load unnecessary data
- Choosing the right strategy depends on performance trade-offs and data access patterns

Entity – Automatic Schema Creation

- ORM frameworks provide automatic schema generation policies based on the defined entities
- These policies define how the ORM should handle the database schema
- Automatic schema creation occurs at application startup, based on the defined entities and the selected policy
- Not all ORMs support every policy, but we can identify a set of the most common

Entity – Schema Creation Policies

- Policies:
 - Create: Generates the schema if it does not exist
 - Drop-and-Create: Deletes and recreates the schema on each run (useful for testing)
 - Update: Modifies the schema to match entity changes (may cause issues with existing data)
 - Validate: Checks if the schema matches the entity definitions but does not modify it
 - None: The ORM does not perform any schema validation or modification
- Choosing the right policy:
 - Development: Create or Drop-and-Create for rapid iteration
 - Production: None or Validate to prevent unintended schema changes

Entity – Testing

- Entities are simple data holders, representing the structure of the database
- They do not contain business logic, only fields, relationships, and metadata (e.g., constraints)
- Testing them is not necessary, as their correctness is enforced by
 - The ORM itself, which ensures mappings are correctly applied
 - The database schema, which enforces constraints (e.g., foreign keys, uniqueness)
 - Other application tests, where entities are implicitly used and validated

Basic querying in ORM

- ORMs generate SQL automatically based on the structure of the defined entities
- Basic queries can be constructed using entity attributes, allowing developers to work at the object level without writing SQL
- Simple queries are fully managed by the ORM (e.g., finding by primary key, inserting, updating, deleting)
- ORMs optimize queries by dynamically translating object-oriented queries into SQL tailored for the underlying database
- Key advantages
 - Minimizes direct SQL interaction, reducing complexity
 - Ensures consistency by enforcing entity relationships and constraints
 - Improves code readability and maintainability by abstracting low-level database interactions

Advanced querying in ORM

- When basic queries are not enough, ORMs offer more advanced techniques:
 - Criteria API & Query Builders – Programmatic query construction
 - Joins and custom queries – Fetching related data efficiently
 - Native SQL queries – Direct SQL execution when necessary
- Performance considerations:
 - Use pagination and indexing to improve query speed
 - Avoid N+1 query problems with proper relationship fetching (e.g., JOIN FETCH, batch fetching)
 - Cache frequently accessed data

ORM- Sessions & Transactions

Session - Definition

A session (or persistence context) is a temporary environment where an ORM tracks and manages entities during their lifecycle. It acts as an intermediary between the application and the database, ensuring that objects remain synchronized with their corresponding records in the database.

Its scope and lifespan must be carefully managed to avoid performance issues.

Session – Key Features

- Manages the identity and lifecycle of entities within an application session
- Keeps track of changes made to managed entities
- Automatically synchronizes data between in-memory objects and the database
- Reduces the need for explicit SQL statements, leveraging automatic persistence
- Provides a caching mechanism, reducing unnecessary database queries
- Ensures consistency within a transaction, committing or rolling back changes

Session - Operations

- Opening a session – Creates a new persistence context
- Persisting an entity – Registers a new entity for saving
- Fetching an entity – Retrieves an entity from the database into the session
- Updating an entity – Modifies an entity already tracked by the session
- Detaching an entity – Stops tracking an entity without deleting it
- Flushing changes – Writes all pending updates to the database
- Closing a session – Ends the persistence context and detaches all entities

Session – Fetching entities

- When an entity is retrieved using an ORM query, it is automatically added to the session, its state is tracked, and any modifications will be synchronized with the database when the session is flushed or committed
- If the same entity is requested again during the session, the ORM will return the existing in-memory instance rather than executing another database query.
- When executing a native SQL query that retrieves data not directly mapped to a known entity, the results are not added to the session. In such cases, the ORM treats the output as raw data, and no automatic tracking or synchronization occurs.

Session – Detaching entities

- Detaching an entity means removing it from the session while keeping it in memory. Once detached, the ORM no longer tracks changes made to the entity, and these modifications will not be reflected in the database unless the entity is reattached to a session.
- Detaching is not removing. Removing an entity means marking it for deletion from the database. It is still tracked by the session until the session is flushed or committed, at which point the corresponding record is physically deleted from the database.
- Detaching only affects the ORM's ability to track changes, while removing permanently deletes the entity's data from the database.
- Detaching can be useful when dealing with large datasets to reduce memory consumption, whereas removing is necessary when an entity is no longer needed in the system.

Short-lived Sessions

- A new session is created for each operation (or web request) and closed immediately after.
- This ensures that entities are not kept in memory longer than necessary.
- Prevents excessive memory consumption and avoids issues such as long-running transactions.
- Recommended for most applications, especially web-based systems where each request is stateless.

Long-lived Sessions

- The session remains open for a longer period, potentially throughout a user session or across multiple operations.
- This allows entities to be reused without repeated database queries.
- However, it increases the risk of memory leaks, concurrency issues, and stale data.
- Can be useful in specific cases, such as batch processing or maintaining user state in desktop applications.

Session – Best practices

- Use short-lived sessions by default unless there is a clear need for long-lived sessions.
- Avoid keeping a session open longer than necessary to reduce memory consumption.
- Flush and commit changes regularly to maintain database consistency.
- Be cautious with lazy loading when using detached entities, as it may trigger unexpected database queries outside of an active session.
- **Modern ORM frameworks manage session automatically**
- Manual session management can be necessary to optimize performances

Transaction - Definition

A transaction is a sequence of one or more database operations that are executed as a single unit of work. Transactions, following the ACID properties, ensure data integrity by guaranteeing that either all operations within the transaction are successfully completed or none of them take effect. This prevents situations where partial updates could leave the database in an inconsistent state.

Transaction – Key Features

- Transactions ensure data integrity by grouping multiple operations together.
- A transaction must be explicitly committed to persist changes or rolled back to undo them.
- If an error occurs during a transaction, the system can abort all changes to maintain database consistency.
- Transactions typically operate within an active session, meaning they are tied to the ORM's persistence context.

Transaction – Operations

- Begin – Starts a new transaction.
- Commit – Saves all changes made within the transaction permanently.
- Rollback – Reverts all changes made within the transaction.
- Savepoint – Creates a checkpoint within a transaction to allow partial rollbacks.
- Isolation Level Configuration– Defines how transactions interact with each other to manage concurrent access to data.

Transaction – Concurrency Issues

- In a multi-user database system, multiple transactions can be executed concurrently. The most common concurrency issues are:
 - Dirty Reads – A transaction reads uncommitted changes from another transaction.
 - Non-Repeatable Reads – A transaction reads the same row twice but gets different values because another transaction modified it in between.
 - Phantom Reads – A transaction executes the same query twice and gets different results because another transaction inserted or deleted rows.

Transaction – Isolation Levels

- Isolation Levels define how transactions interact with each other and how data consistency is maintained in concurrent operations.
- A lower isolation level increases performance but allows more concurrency issues, while a higher isolation level ensures stronger data integrity but may reduce performance.

Transaction – Isolation Levels

- Read Uncommitted
 - Transactions can read uncommitted changes from other transactions (dirty reads allowed).
 - Highest concurrency, lowest data consistency.
- Read Committed
 - A transaction can only read committed changes (no dirty reads).
 - However, non-repeatable reads and phantom reads can still occur.
- Repeatable Read
 - Prevents dirty reads and non-repeatable reads by ensuring that a row read within a transaction does not change until the transaction completes.
 - Phantom reads may still occur.
- Serializable
 - The strictest level: transactions are fully isolated from each other by locking rows or entire tables.
 - Prevents all concurrency issues but significantly reduces parallelism.
- Isolation level depends on the application's performance and consistency requirements.
 - High-performance application: Read Committed
 - Data-critical applications: Serializable

Transaction – Best Practices

- Keep transactions as short as possible to avoid locking resources for too long.
- Always handle rollback scenarios to prevent data inconsistencies in case of failures.
- Use proper isolation levels based on the application's concurrency requirements.
- **Modern ORM frameworks manage transactions automatically**
- Manual transaction management can be necessary to optimize performances

Entity - Lifecycle

- Entities transition through different states during their lifecycle in an ORM
- Main entity states:
 - Transient – The entity is created in memory but not yet persisted
 - Persistent – The entity is associated with a database session and will be saved
 - Detached – The entity is no longer managed by the ORM but still exists in memory
 - Removed – The entity is marked for deletion and will be removed from the database

Entity – Lifecycle operations

- These operations modify the state of an entity, when performed
- Lifecycle operations:
 - Creating a new instance (e.g. `new Entity()`) starts in Transient state
 - Persisting the entity (e.g., `save()` or `persist()`) transitions it to Persistent
 - Closing or clearing the session detaches the entity (Detached)
 - Deleting the entity moves it to the Removed state

The Repository Pattern

Repository – Definition

The Repository Pattern is an architectural pattern that provides an abstraction layer between the application's business logic and the database.

It defines a dedicated component responsible for retrieving, persisting, and managing entities, ensuring that data access operations are encapsulated and decoupled from other parts of the system.

Repository – Key Features

- Acts as an intermediary between the ORM and business logic.
- Encapsulates data access logic within a dedicated component.
- Keeps database operations separate from business logic.
- Provides a structured interface for querying and modifying entities, improving code maintainability.
- Centralizes data access in one place, reducing code duplication and enforcing consistency.
- Supports standard data access patterns, including CRUD operations, query methods, and custom queries.
- Improves testability by abstracting persistence, making it easier to mock DB interactions or repositories entirely.

Repository – CRUD Operations

- Repositories provide a standard set of methods to manage entities in the database.
- The four primary operations in a repository are:
 - Create – Inserts a new entity into the database.
 - Read – Retrieves entities by ID or search criteria.
 - Update – Modifies an existing entity and saves changes.
 - Delete – Removes an entity from the database.
- Using a repository for CRUD operations ensures a consistent API across the application.

Repository – Basic Queries

- Repositories leverage ORM capabilities to query the database using entity attributes
- Query methods allow filtering and retrieving entities based on their attributes, reducing the need to write explicit SQL
- Queries can be composed dynamically, combining multiple conditions within repository methods
 - `findByNameAndEmail (name, email)`: Finds an entity based on its name and email attribute.
 - `existsByEmail(String email)`: Checks if an entity with a given email exists
- Using repository query methods ensures cleaner and more maintainable business logic, keeping data access centralized

Repository – Custom Queries

- When query methods are not enough, repositories can define custom query methods
- Custom query methods are useful for
 - Complex filtering with advanced multiple conditions
 - Aggregation queries (e.g., count, sum, averages)
 - Joins and nested queries involving multiple entities
- Custom queries can be written using
 - ORM query languages (e.g., JPQL, TypeORM Query Builder)
 - Native SQL queries, for performance optimizations
- While custom queries provide flexibility, they should be used carefully to maintain abstraction
 - If the result structure of a custom query does not match an entity, raw data (tuples) may be returned instead, causing a loss of abstraction

Repository – Pagination & Sorting

- When handling large datasets, repositories provide mechanisms for efficient pagination and sorting.
- Pagination: Limits the number of results returned per request to improve performance.
 - Example strategies: limit/offset, cursor-based pagination.
- Sorting: Orders results based on specified fields.
 - Example: `findAll(Sort.by("name").ascending())`.
- Proper use of pagination and sorting prevents excessive memory consumption and speeds up query execution.

Repository – Testing

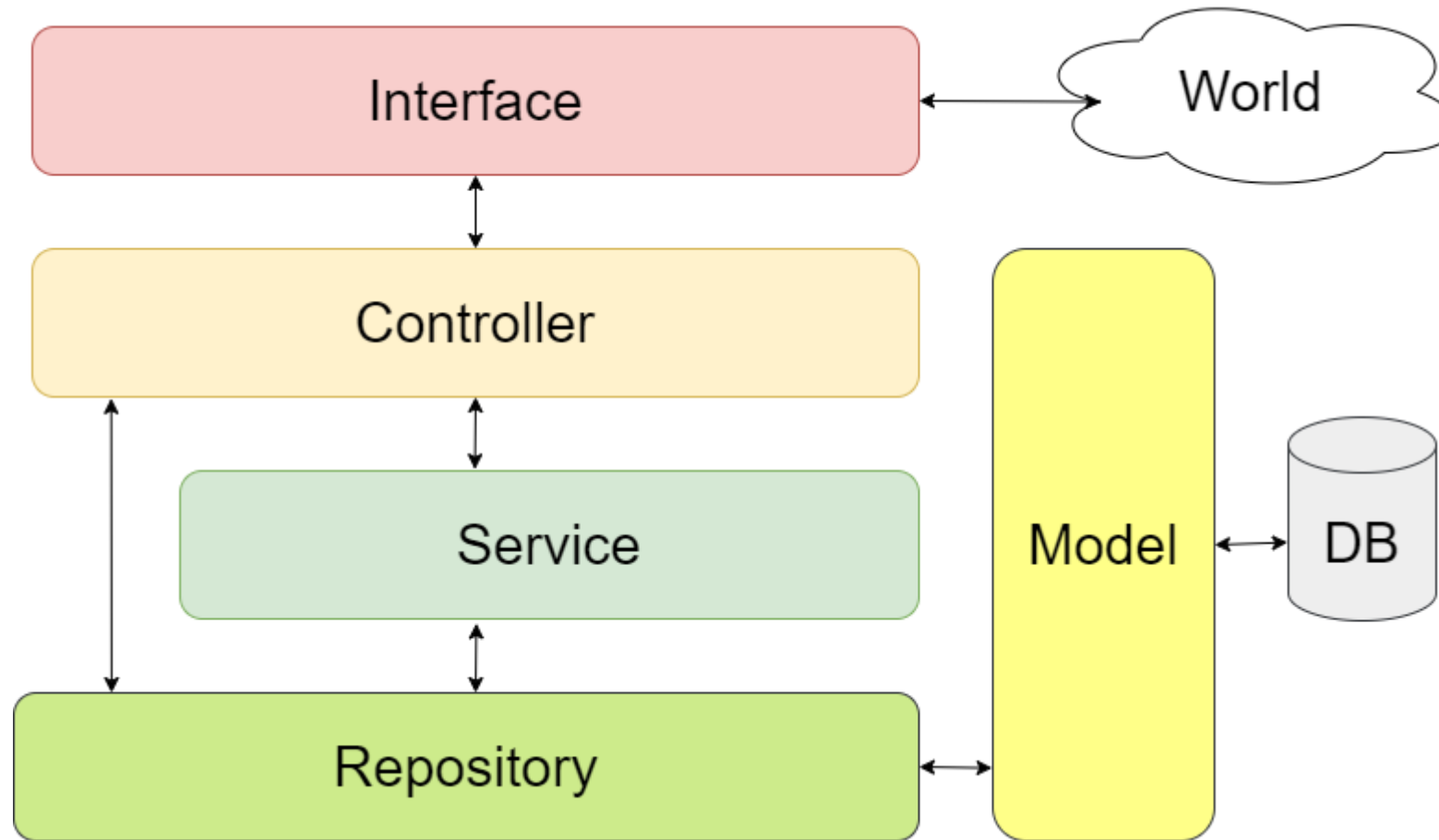
- A repository should be tested at different levels while ensuring that database interactions are always mocked.
- Unit Testing the Repository
 - Standard ORM operations (save(), findById(), delete()) should be mocked, ensuring the repository correctly interacts with the ORM.
- Unit Testing Higher-Level Components
 - Repository itself, if involved, should be mocked entirely.
- Integration Testing the Repository
 - The repository itself should not be mocked
 - All its internal ORM operations (save, delete, find) must be mocked.

Designing a Layered ORM-Based Application

Layered Structure - Definition

- A layered architecture is a software design pattern that organizes an application into multiple, distinct levels, each with a specific responsibility.
- Each layer communicates only with adjacent layers, maintaining a structured and predictable flow of data.
- This structure enforces a clear separation of concerns, preventing different parts of the application from becoming tightly coupled.

ORM-based Layered Structure – Example



ORM-based Layered Structure – Key Features

- Encapsulates responsibilities, making the application modular and maintainable.
- Decouples concerns, preventing business logic from mixing with database operations.
- Enhances testability, allowing independent testing of each layer.
- Facilitates scalability, as layers can be modified independently.
- Improves security, ensuring controlled access to data through repositories.

ORM-based Layered Structure – Layers

- Interface Layer
 - Manages external interactions (HTTP, API, message queues).
- Controller Layer
 - Aggregates data, orchestrates repositories and services.
- Service Layer (Business Logic)
 - Implements core logic (data transformation, calculations, specific domain operations).
- Repository Layer
 - Encapsulates database access, interacts with ORM.
- Model Layer
 - Defines entities and relationships, representing the application domain.

Design - Best practices

- Follow the layered approach to achieve Separation of Concerns.
- Do not access ORM outside of repositories.
- Use services for non-trivial business logic, keeping controllers lightweight.
- Optimize entity relationships (eager vs. lazy loading) to prevent unnecessary queries.
- Leverage transaction management to ensure data consistency.
- Use repository methods instead of raw SQL, unless optimization is needed.

Design - Pitfalls to Avoid

- Skipping layers arbitrarily, leading to inconsistent architecture.
- Placing business logic in controllers, making them too complex.
- Overloading repositories with business logic, reducing their clarity.
- Ignoring pagination and filtering, leading to performance issues.
- Hardcoding queries, making maintenance harder.
- Neglecting testing at different levels, increasing debugging difficulty.